

Universidade Leiria • Oeste
Escola de Tecnologia e Gestão

Desenvolvimento de Aplicações Distribuídas

JavaScript

JavaScript Fundamentals

Marco Monteiro and Ricardo Gomes

1. Basic concepts and syntax
2. Objects
3. Functions and functional programming
4. Asynchronous programming
5. Modules and Bundlers
6. DOM API
7. TypeScript

Basic concepts and syntax

Basic concepts and history

- JavaScript is the de facto standard language of Web Browsers
  - Browsers include JavaScript engines that allows JavaScript to be executed natively - without plugins, external runtimes, or compilation.
  - TypeScript (and other languages like JSX, Dart or CoffeeScript) are transpiled into JavaScript to run in browsers.
  - Modern browsers also support WebAssembly (Wasm), a low-level, fast, and portable binary format. Other languages are compiled into Wasm and executed in the browser
- JavaScript can run outside the browser using execution environments that include JavaScript engines, most notably Node.js, but also Deno or Bun.
- JavaScript was created in 1995 by *Brendan Eich* at Netscape and has evolved continuously. Since 1997, it follows the ECMA standard and is officially called ECMAScript. The latest version is ECMAScript 2025, 16th edition.
  - Check latest JavaScript features at: https://caniuse.com/

JavaScript characteristics

- JavaScript is an interpreted language.
  - However, modern JavaScript Engines employ just-in-time (JIT) compilation to optimize execution.
- The language is dynamic and loosely typed.
  - Type checking occurs at runtime only. Implicit type conversions (coercions) occur between different types, often producing results that may be unexpected.
- JavaScript supports multiple programming paradigms:
  - Imperative programming.
  - Object-Oriented programming, primarily prototype-based (with class-based syntax provided as syntactic sugar).
  - Functional programming, as functions are treated as first-class citizens.
- Syntax is derived from the C language family (C, C++, Java, C#, etc.).

Basic syntax

- JavaScript's basic syntax is similar to C, C++, Java, and C#.

```
//comments          {…block…}          if          switch
while               do…while           try…catch…finally
```

- We will emphasize on key syntax differences

- Variables and constants

```
a       = 1  // Not recommended
var b   = 2  // Function scope
let c   = 3  // Block scope – Recommended
const d = 4  // Recommended for constants
```

- Loops - for:

for .. of

Loops over elements of an iterable

(Array, String, Map, Set, etc.)

```
for (let i = 0; i < 10; i++) {
   console.log(i)
}

let array = ['A', 'B', 'C'];
for (let element of array) {
   console.log(element)
}
```

Operators

- JavaScript basic operators are similar to C, with some additions
  - strict equality / strict inequality: `===` `!==`

```
a === b // strict equality   - true if type and value are equal
a !== b // strict inequality - true if type or value are not equal
```

```
a ==  b // loose equality    - true if value is equal
a !=  b // loose inequality  - true if value is not equal
```

  - nullish coalescing operator: `??`

```
const w = a ?? 'value when a is null or undefined' //nullish coalescing
```

```
const x = a || 'value when a is falsy'                                  // or
const y = a && 'value when a is truthy'                                 // and
const z = a ? 'value when a is truthy' : 'value when a is falsy' // ternary
```

  - optional chaining: `?.`

```
const city = person?.address?.city                  // optional chaining
    // if any property in the chain is null or undefined,
    // expression returns undefined (instead of throwing an error)
const cityName = person?.address?.city ?? 'Unknown'
    // cityName = 'unknown',  if any property is null or undefined
```

Data types – primitive types

string	Strings are immutable. There is no character type. e.g., `'abc'` `"def"` `'a'` `"b"` `''` `""` **Template Literals** are delimited by backticks (`) e.g., `` `Hello ${person.name}, welcome!` ``
number	All numbers, including integers, are handled as floats. e.g., `34.89` `2` `2.0` (`2 == 2.0`)
bigint	Introduced on ES2020 (ES11) to handle integers only (solves precision problems with numbers). e.g., `34n`
boolean	`true` or `false` . In expressions, other values may be treated as `truthy`: (e.g., `1`, `0.001`, `"text"`, `' '`) or `falsy`: (`0`, `-0`, `0n`, `''`, `""`, `null`, `undefined`, `NaN`).
null	Value that represents empty/nothing
undefined	The absence of value (variable/property is declared but no value is assigned to it)

Data types – composite types

object	Objects are collections of properties (key–value pairs) e.g., `{}` `{propertyA: 'a', propertyB: 2}` Methods: properties whose values are functions Prototype-based inheritance
array	Ordered, zero-indexed collection of values (of any type) e.g., `[]` `[2, 'a', {a: 3}, "b", null]` Arrays are specialized **objects** with numeric keys (indexes). Also, as objects they have properties and methods: e.g., `[1, 'a'].length` `[1, 'a'].push('b')`
function	Functions are first-class citizens. This means functions can be: - Assigned to variables - Passed as arguments - Returned from other functions - Stored in objects/arrays Functions are also **objects** (with properties).
built-in objects	JavaScript includes many other built-in objects - examples: `Error`, `Math`, `Date`, `RegExp`, `Map`, `Set`, `JSON`, `Promise`

Arrays

- Examples of code:

```
let a = [1, 'a']
```

==

```
let a = new Array(1, 'a')
```

```
let a = ['a', 'b', 'c']
console.log(a[0])     // a
console.log(a.length) // 3
a.push("d")   // adds a new element to the array
              // a === ['a', 'b', 'c', 'd']
a.shift()     // removes the first element
              // a === ['b', 'c', 'd']
a.pop()       // removes the last element
              // a === ['b', 'c']
```

Check: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

Arrays

- Transverse the array: **for .. of** loop

```
let a = ['a', 'b', 'c']
for (let element of a) {
  console.log(element) // 'a', 'b', 'c'
}
```

- Transverse the array: **foreach** (callback)

```
let a = ['a', 'b', 'c']

a.forEach(element => {
  console.log(element) // 'a', 'b', 'c'
})

a.forEach((element, index) => {
  console.log(index)   // 0, 1, 2
  console.log(element) // 'a', 'b', 'c'
})
```

The argument passed to **forEach** is a callback function, which is executed once for each element in the array.

Map and Set

- **Map** – iterable collection of **key-value** pairs

```
let m = new Map()
m.set('key1', 1)
m.set(2, 'value2')

for (let value of m.values()) {
  console.log(value)   // 1, value2
}

for (let key of m.keys()) {
  console.log(key)     // key1, 2
}

m.forEach((value, key) => {
  console.log('m[' + key + ']=' + value)
              // m[key1]=1, m[2]=value2
})
```



```
1
value2
key1
2
m[key1]=1
m[2]=value2
```

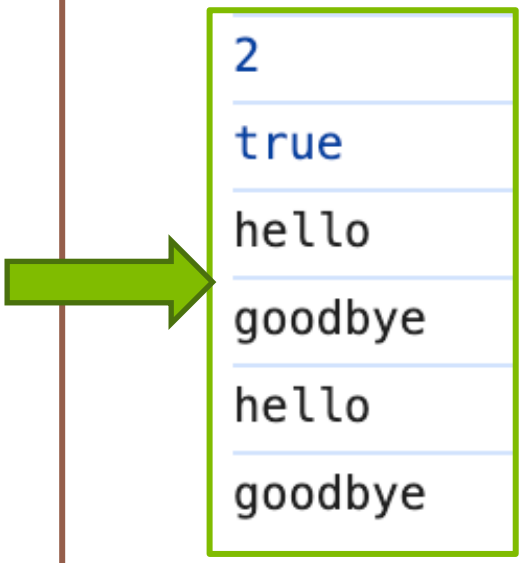
Map and Set

- **Set** – iterable collection of **unique** values

```
let s = new Set()
s.add("hello").add("goodbye").add("hello")
console.log(s.size)             // 2
console.log(s.has("hello"))     // true

for (let v of s.values()) {
  console.log(v)      // 'hello', 'goodbye'
}

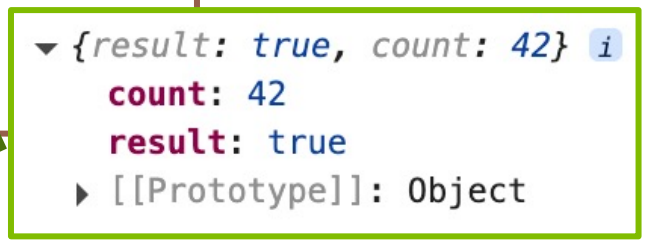
s.forEach(value => {
  console.log(value) // 'hello', 'goodbye'
})
```



JSON

- JSON – convert values to/from JSON
  - parse– converts a JSON string to a JavaScript value

```
const jsonStr = '{"result":true, "count":42}';
const obj = JSON.parse(jsonStr);
console.log(obj)
```



  - stringify– converts a JavaScript value to a JSON string

```
let myCar = { color: 'red',
  equipment: ['GPS', 'ABS', 'Bi-Xenon'],
  engine: { cylinders: 4, size: 2.2 }
}
const jsonStr = JSON.stringify(myCar);
console.log(jsonStr)
```

```
{"color":"red","equipment":["GPS","ABS","Bi-Xenon"],"engine":{"cylinders":4,"size":2.2}}
```

RegExp

- RegExp– matching text with patterns
  - Use patterns to search, validate, filter, replace, split strings
  - Example to validate (using the test method):

```
const re1 = /^[1-9][0-9][0-9][0-9]-[0-9][0-9][0-9]$/
const re2 = new RegExp('^\\d{4}-\\d{3}$')
const validCode1 = re1.test('0222-022')
const validCode2 = re2.test('0122-022')
console.log(validCode1)
console.log(validCode2)
```



```
false
true
```

Objects

Object

- A JavaScript object is a **collection of properties** (key–value pairs)
  - If a property value is a function, it is known as a **method**
- **Prototype based inheritance**
  - Objects reuse behavior through a shared prototype chain (by delegation)
  - **No class hierarchy** - objects inherit directly from other objects
  - ES6 (ES2015) adds class-based syntax (like Java), but this is a syntactic sugar - under the hood, it uses the prototype chain.
- Extending JavaScript Objects
  - **Dynamic properties** – Add, modify, or delete properties at runtime
  - This is different from class-based languages, where the class structure is usually fixed at compile-time.

Object literals

- Create object instances using object literals
  - Key-value pair syntax (where key is the property name):

```
property_name: value
```

```
let emptyObj = {} // empty object

let person = {
  firstName: 'John',
  familyName: 'Doe',
  age: 30,
  displayName: function() {
    console.log(this.firstName + ' ' + this.familyName)
  }
}
person.displayName()
```

```
John Doe
```

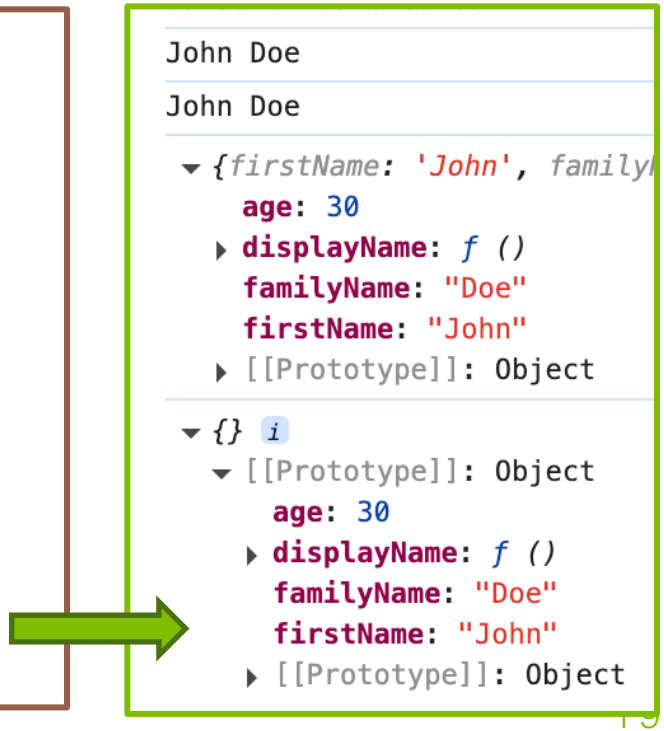
  - **this** – reference to the object instance

Object.create

- Create object instances using another object as prototype
  - Sets the new object's prototype to the specified object; does not copy properties.
  - The new object inherits behavior via the prototype chain (new object delegates to the prototype)

```
let person = {
  firstName: 'John',
  familyName: 'Doe',
  age: 30,
  displayName: function() {
    console.log(this.firstName + ' ' + this.familyName)
  }
}

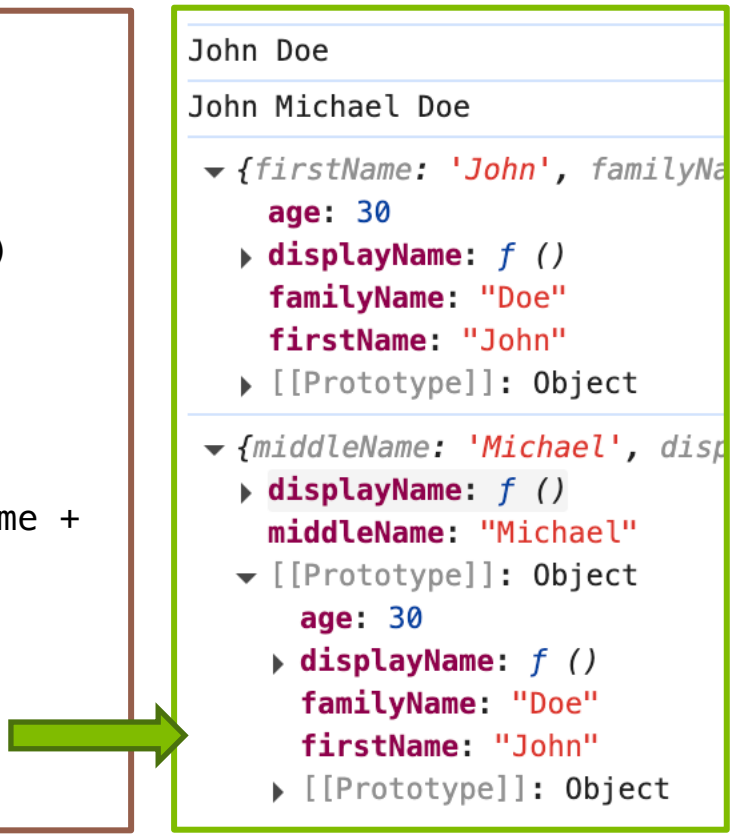
let newPerson = Object.create(person)
person.displayName()
newPerson.displayName()
console.log(person)
console.log(newPerson)
```



Extending objects

- Objects are extended by adding properties

```
let person = {
  firstName: 'John',
  familyName: 'Doe',
  age: 30,
  displayName: function() {
    console.log(this.firstName + ' ' + this.familyName)
  }
}
let newPerson = Object.create(person)
newPerson.middleName = 'Michael'
newPerson.displayName = function(){
  console.log(this.firstName + ' ' + this.middleName +
              ' ' +   this.familyName)
}
person.displayName()
newPerson.displayName()
console.log(person)
console.log(newPerson)
```



- newPerson object has 2 new property: `middleName` and `displayName`
- When multiple properties have the same name (displayName) along the prototype chain, the own property on the object (the one just created) takes precedence over the prototype's property.

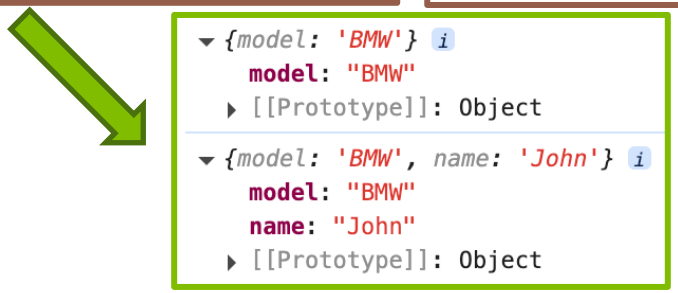
Cloning objects

- Simple shallow copy. Copies only own properties.
  - Object Spread: `{ ...obj }`
  - `Object.assign()`

```
let car = { model: BMW' }
let o = { name: 'John' }
let car2 = { ...car }
//Merge multiple source objects.
let mergedObj = { ...car, ...o}

console.log(car2)
console.log(mergedObj)
```

```
let car = { model: 'BMW' }
let o = { name: 'John' }
let car2 = Object.assign({}, car)
//Merge multiple source objects.
let mergedObj =
    Object.assign({}, car, o)
console.log(car2)
console.log(mergedObj)
```



- Deep copy. Copy include nested objects, arrays, Maps, Sets, etc.
  - `structuredClone()`

```
const deepCopy = structuredClone(originalObj)
```

Functions and functional programming

Function syntax

- Classic syntax:
  - Named function

```
function add(x, y) {
  return x + y
}
const a = add
console.log(a(1, 2))  // 3
```

  - Anonymous function

```
const a = function (x, y) {
  return x + y;
}
console.log(a(1, 2))  // 3
```

- Arrow function
  - Anonymous functions with "lighter" syntax:

```
const a = (x, y) => x + y
console.log(a(1, 2))  // 3
```

- Classic: "**this**" keyword may refer to the function itself or to the context where it was called from.
- Arrow function: "**this**" keyword maintains its lexical scope.

Check: https://developer.mozilla.org/docs/Web/JavaScript/Reference/Operators/this

Arrow functions

left side - **arguments**

```
(x, y) => x + y
```

right side – **body** of function

- **Expression** – function returns the expression value
- **Block `{...}`** – normal function block

- No arguments

```
() => 3.14
```

- One argument

```
a => 3 * a
```

```
(a) => 3 * a
```

- Multiple arguments

Body is an expression that is returned automatically

```
(a, b) => a * b
```

These two functions do the same

Body is a "normal" block

```
(a, b) => {
  return a * b
}
```

- With blocks:

```
(a) => {
  console.log(a)
  return a * 2
}
```

Arrow functions – declaration and invoking

- Arrow functions are anonymous and can be invoked either by assigning them to a reference (variable, constant, property, etc.) or by calling them directly inline.

- Function declaration:

```
(x, y) => x + y
```

- Invoking from a constant:

```
const a = (x, y) => x + y
let res = a(1, 2)  // res == 3
```

a constant value is a function

- Inline invoking:

```
let res = ( (x, y) => x + y )(1, 2)  // res == 3
```

Functional programming

- Functions are first-class citizens. It is possible to:
  - Assign functions to variables, constants, object properties (called methods) or store them in collections
  - Pass a function (called the callback function) as an argument to another function.
  - Return a function from other functions. Returned function is called a closure when it retains variables or constants from the outer function
- Functions are also objects, with their own properties
  - It is possible to extend a function object by adding our own properties
- These characteristics allow JavaScript to support Functional Programming concepts
  - JavaScript is not a pure functional language (*it mixes functional programming concepts with imperative and OOP*)

Nested functions and closures

- Nested functions:

```
const outer = () => {
  console.log("Outer")
  const inner = () => {
    console.log("Inner")
  }
  inner()
}
outer()
```

- Closure:

```
const outer = () => {
  let i = 1
  console.log("Outer")
  return () => {
    console.log("Inner")
    console.log(i)
    i++
  }
}
const closureFunction = outer()
closureFunction()
closureFunction()
```



```
Outer
Inner
1
Inner
2
```

- Closure function is a function returned by the outer function, that retains variables or constants from the outer function

Callbacks

- A callback function is a function that is passed as an argument to another function, which can then invoke it. Examples:
  - Event handler

```
btn.addEventListener('click', () => {
  // callback function
  console.log('Click')
})
```

  - Handling the response from the API

```
axios.get(url)
  .then(response => {
    // callback function
    console.log(response)
  })
```

  - Manipulating arrays using functional programming – with callbacks

```
const a = [1, 2, 3, 4]
a.forEach( element => {
  console.log(element)
})
```

```
const a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
const b = a.filter( v => v % 2 === 0 )
console.log(b)
```



  - Other array methods that accept callbacks: map, every, some, find, reduce, etc.

Check: https://developer.mozilla.org/docs/Web/JavaScript/Reference/Global_Objects/Array

Asynchronous programming

Asynchronous programming

- Asynchronous Programming – A programming model where tasks run in the background without blocking the main thread, so other operations can continue until results are ready.
- Synchronous vs Asynchronous
  - Synchronous (classic model): tasks run in a strict sequence; each instruction waits for the previous one to finish.
  - Asynchronous: tasks can be initiated and completed at different times; execution continues without waiting, and completion order is not guaranteed.
- Typical use in JavaScript web apps:
  - Calling API endpoints without blocking the UI (network requests are slow compared to most app operations).

Promises

- Modern JavaScript implements asynchronous programming with Promises. What is a Promise?
  - Represents the result of an asynchronous operation that will complete in the future (success or failure).
  - Code continues running without blocking; the Promise notifies when the result is ready.
  - Analogy: Ordering food at McDonald's — you get a receipt (Promise) immediately, and the meal (result) arrives later.
- Promises are implemented by the native Promise class.
  - Libraries like Axios or Fetch create and return a native Promise.
  - When using these libraries, we are consuming Promises already created by them; manual creation of Promises is not needed.

Examples of asynchronous programming

- Promises can be handled in two alternative syntaxes: chaining (`then/catch`) or `async/await` — both rely on Promises internally.
- Asynchronous code to call the endpoint "api/data" using axios library:
  - Chaining:

```
console.log('A')
axios.get('/api/data')
  .then(response => {
    console.log(response.data)
  })
  .catch(error => {
    console.error(error)
  })
console.log('B')
```

```
A
B
Data from the server
```

  - Async/await:

```
async function fetchData() {
  try {
    console.log('A')
    const response = await axios.get('/api/data')
    console.log(response.data)
    console.log('B')
  }
  catch (error) {
    console.error(error)
  }
}
fetchData()
```

```
A
Data from the server
B
```

Sequence of operations

- A sequence of operations is when multiple asynchronous tasks must be executed one after the other, where each step depends on the result of the previous one (e.g., fetch user → then fetch user's posts).
- Common pitfalls:
  - Independent chains: multiple .then blocks that run separately instead of sequentially.
  - Callback nightmare (historical context): deeply nested callbacks used before Promises existed, making code hard to read and maintain.
- Correct approach:
  - Use Promise chaining or async/await to ensure operations run in order.

```
axios.get('/api/first')
  .then(res1 => {
    console.log(res1.data)
    // chain return:
    return axios.get('/api/second')
  })
  .then(res2 => {
    console.log(res2.data)
  })
.catch(error =>
    console.error(error)
)
```

```
async function fetchData() {
  try {
    const res1 = await axios.get('/api/first')
    console.log(res1.data)
    const res2 = await axios.get('/api/second')
    console.log(res2.data)
  } catch (error) {
    console.error(error)
  }
}
fetchData()
```

Parallel API Calls – Sequential Await Pitfall

- Problem: Independent API calls are often written with sequential await, causing unnecessary delays. Example of the problem:

```
async function fetchData() {
  const products = await axios.get('/api/products')
  const cities = await axios.get('/api/cities')
  console.log(products.data, cities.data)
}
fetchData()
```



- Solution: Run calls in parallel using Promise.all:

```
async function fetchData() {
  const [products, cities] = await Promise.all([
    axios.get('/api/products'),
    axios.get('/api/cities')
  ])
  console.log(products.data, cities.data)
}
fetchData()
```

Version with "chain" syntax:

```
Promise.all([
  axios.get('/api/products'),
  axios.get('/api/cities')
])
.then(([prods, cits]) => {
  console.log(prods.data, cits.data)
})
.catch(error => console.error(error))
```

Modules and Bundlers

Modules

- ES Modules (ESM) were introduced in ES6 (ES2015).
- Modern browsers load modules by referencing a JavaScript file in HTML with the attribute `type="module"`

```
<script type="module" src="main.js"></script>
```

- Module files use **export** statements to define which values or functions are available to other modules, and **import** statements to bring in values or functions from other modules

- module.js:

```
export const pi = 3.141593
export const sum = (x, y) => x + y
export default {
  multiply : (x, y) => x * y
}
```

- File that uses module.js:

```
// import default as m
import m from './module.js'
import { pi, sum } from './module.js'
console.log(sum(2, 3))
console.log(m.multiply(pi, 3))
```



JavaScript Bundler

- A JavaScript bundler is a tool that takes many small files (modules) and combines them into a single, optimized file that runs efficiently in the browser.
- Why are JavaScript bundlers needed?
  - Legacy browsers do not support ES Modules consistently.
  - Large apps may use hundreds of modules → slow and inefficient if loaded one by one.
  - Bundlers apply crucial optimizations:
    - Transpiling: convert TypeScript, JSX, etc. → JavaScript.
    - Minification: reduce file size for faster loading.
    - Asset management: handle CSS, images, and other resources.
  - Crucial for **production builds**, where performance and compatibility are critical.
- Examples:
  - Webpack; Parcel; Rollup; **Vite** (used in the practical classes and projects of this course)

Vite

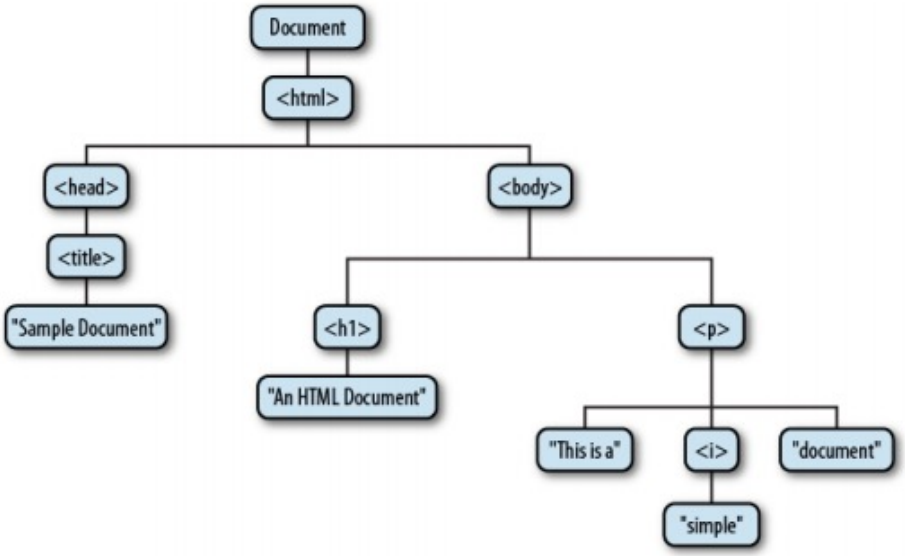
- A fast, modern build tool that leverages native ES Modules for instant development and optimized production builds.
  - The goal is to combine the best of both worlds: leveraging the browser's native modules to avoid rebuilding the app during development, while producing an optimized application with a traditional JavaScript bundler for production.
- During development:
  - Uses native ES Modules – loads code directly in the browser without full bundling, speeding up development.
  - Hot Module Replacement (HMR) – updates only the changed code in the browser instantly, so you see edits immediately without a full page reload.
  - Framework support – Vue, React, Svelte, and more.
- Production Build:
  - Optimized bundles – uses Rollup under the hood.
  - Minification & tree-shaking – smaller, efficient files.

DOM API

Document Object Model

Document Object Model (DOM)

- Document Object Model (DOM) is a tree-like representation of a web page's structure that allows programs to read and manipulate content, structure, and style



- The DOM is primarily manipulated through JavaScript. WebAssembly itself does not have direct access to DOM - it must call JavaScript functions to interact with the DOM
- The DOM is not part of the JavaScript language — it is a Web API provided by browsers, but it is fundamental for client-side web programming.
  - Node.js, Deno or Bun do not have access to the DOM

Document Object Model (DOM)

- The DOM is a very large API; we will focus only on the fundamentals.
  - DOM Reference: https://developer.mozilla.org/docs/Web/API/Document_Object_Model
- Core DOM Interactions
  - **Selection** – accessing elements (`getElementById`, `querySelector`, etc.)
  - **Manipulation** – modifying content, attributes, styles, or document structure (`innerHTML`, `classList`, `style`, etc.)
  - **Events** – responding to user interactions (`addEventListener`, `onclick`, etc.)
- Modern frameworks (React, Vue, Angular…) encapsulate DOM usage.
  - Direct DOM manipulation is rare in modern web apps, similar to how game developers rarely call low-level Windows graphics APIs directly — they use engines like Unity or Unreal.
  - Basic DOM knowledge is useful for small apps and minor page adjustments.

DOM - Selection

- Selecting elements in the DOM is the first step to read or manipulate them.

Example:

```
// Select a single element by ID
const header = document.getElementById('header');

// Select the first element that matches a CSS selector
const firstItem = document.querySelector('.item');

// Select all elements that match a CSS selector
// Returns a NodeList which can be iterated over
const allItems = document.querySelectorAll('.item span');
```

Other examples of selection methods:

- getElementsByClassName('className') – elements by class
- getElementsByTagName('tagName') – elements by tag (e.g., `div`)
- getElementsByName('name') – elements by name

Examples of navigation properties:

- node.parentNode – parent element
- node.childNodes – all child nodes (including text nodes)
- node.children – child elements only
- node.firstChild / node.lastChild – first or last child element

DOM - Manipulation

- Manipulating elements in the DOM allows changing content, attributes, styles, or the structure of a web page (add, remove or replace nodes)

Example:

```
const link = document.getElementById('myLink');
// Change content
link.textContent = 'Visit Example';
// Change an attribute
link.href = 'https://example.com';
// Add and remove a class
link.classList.add('highlight');
link.classList.remove('hidden');
```

Other manipulation methods/properties:

- element.innerHTML – get or set HTML content inside an element
- element.setAttribute(name, value) / getAttribute(name) – set or get attributes
- element.appendChild(node) – add a child node
- element.removeChild(node) – remove a child node
- element.replaceChild(newNode, oldNode) – replace a child node

DOM - Events

- Events in the DOM allow code to respond to user interactions or other changes on a web page.

Example:

```
// Page load event
window.addEventListener('load', () => {
    console.log('Page fully loaded');
});
const button = document.getElementById('myButton');
// Respond to a click
button.addEventListener('click', () => {
   console.log('Button clicked!');
});
// Respond to mouse over
button.addEventListener('mouseover', () => {
    button.style.backgroundColor = 'lightblue';
});
```

TypeScript

TypeScript Fundamental Concepts

What is TypeScript & Why use it?

- TypeScript is a **typed superset** of JavaScript:
  - Developed and maintained by Microsoft (open source).
  - Every valid JavaScript code is valid TypeScript code (.js → .ts).
  - Adds **static type checking** on top of JavaScript's dynamic runtime.
- Key Benefits:
  - Catch errors early: Detect type mismatches, typos, and undefined property accesses at compile-time (before running the code).
  - Developer Experience & Tooling: Autocomplete (IntelliSense), confident refactoring, and self-documenting code.
  - Team collaboration: Makes contracts and models explicit across frontend components and API responses.
- Adoption in modern stacks:
  - First-class support in modern frameworks: React, Vue 3, Vite, Pinia, NestJS, etc.

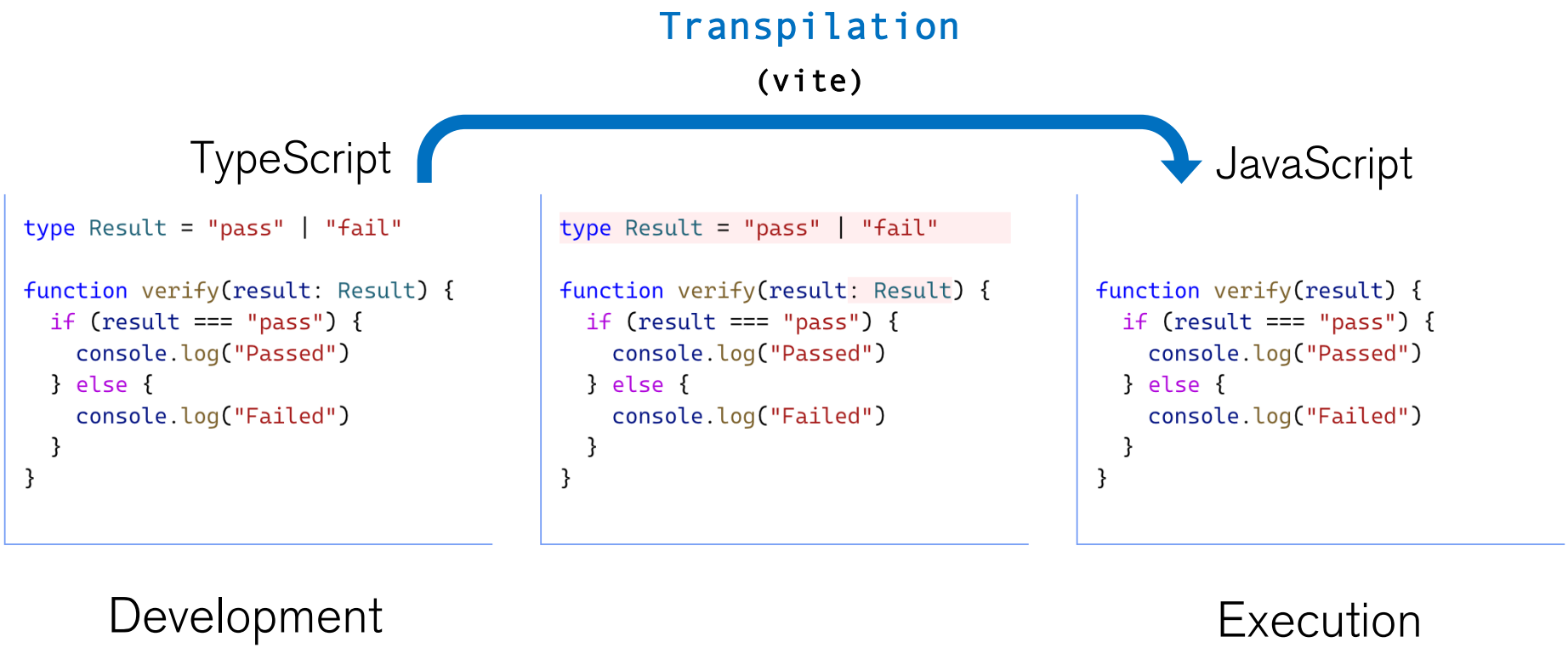
Transpilation & Execution Model

- Browsers and runtimes (e.g., Node.js) **only execute JavaScript**. They cannot run TypeScript directly.
  - Consequently, TypeScript must be **transpiled** into plain JavaScript *(transpilation is source-to-source compilation between languages at a similar abstraction level)*.
  - Modern build tools like Vite handle transpilation seamlessly during development
  - The transpiler strips away all types, interfaces, and annotations to emit clean JavaScript.
  - Static Type Checking: Type errors and mismatches halt the transpilation process, enforcing static type safety before any code can run.
  - Zero runtime overhead: Types exist solely during development and build time; execution performance is identical to plain JavaScript.

Note: Currently, some runtimes (e.g., Deno, Bun) can execute TypeScript directly – however, under the hood, they transpile TypeScript to JavaScript just before execution.

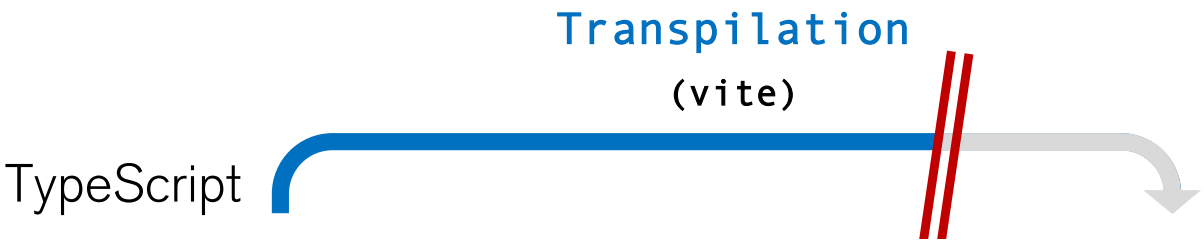
Transpilation & Execution Model

- Transpilation - Strips Types



Transpilation & Execution Model

- Transpilation – Static Type Checking




```typescript
type Result = "pass" | "fail"
function verify(result: Result) {
  if (result === "pass") {
    console.log("Passed")
  } else {
    console.log("Failed")
  }
}
verify("invalid")
// This will cause a TypeScript error
// because "invalid" is not assignable to type Result
```

Transpilation aborted



No JavaScript is emitted

Basic Types & Type Inference

- Explicit Type Annotations
  - **Primitive types** that mirror JavaScript: **string**, **number**, **boolean**, **null**, **undefined**, **bigint**
  - **Arrays**: it is possible to specify the type of an array – e.g., **number[]**
  - **Special types**: **any** (no type checking), **void** (value of functions that return no value)
  - **Union types**: specific set of types or literal values

```
let studentName: string = 'Maria';
let age: number = 21;
let isEnrolled: boolean = true;
let scores: number[] = [16, 18, 14];
// Union types:
let responseCode: number | 'unknown' | 'timeout';
let userId: string | number | null;
```

- Type Inference
  - Types can be inferred from values

```
let x = 'DAD'; // Inferred as string
x = 10; // Error: Type 'number' != 'string'
```

Objects, Interfaces & Custom Types

- Like variables, object properties and function parameters can also be type-constrained:
  - **Optional** properties and parameters (**?**) — elements that are not mandatory.

```
function printCoord(pt: { x: number; y: number; info?: string /* info is optional */ }) {
  console.log(`(${pt.x}, ${pt.y})`);
}
printCoord({ x: 3, y: 7 });
```

  - Interfaces and Custom Types
    - **interface** and **type** define the structure and data types of object properties.
    - Interfaces can extend other interfaces to reuse properties.
    - Types support unions, type aliases and tuples.

```
interface User {
  id: number | string;
  name: string;
  avatarUrl?: string; // Optional property
}
const user: User = { id: 1, name: 'John Doe', };
```

```
type Role = 'admin' | 'player';
type GamePlayer = {
  role: Role;
  score: number;
}
const player: GamePlayer = {
  role: 'player',
  score: 100,
};
```

Functions & Generics

- Typed Functions (standard & arrow functions)
  - Parameter types are explicit; return types can be explicit or inferred.
  - Supports optional (?) and default parameters.

```
function calculateTotal(price: number, tax: number = 0.23): number {
  return price * (1 + tax);
}
```

```
const formatUser = (name: string, role?: string): string => {
  return role ? `${name} (${role})` : name;
};
```

- Generics (<T>)
  - Enables writing reusable functions and structures that support multiple types while preserving strict type safety.

```
function getFirstElement<T>(items: T[]): T | undefined {
  return items[0];
}
const num = getFirstElement([10, 20, 30]); // Inferred as number | undefined
const str = getFirstElement(['a', 'b']) // Inferred as string | undefined
```

Other Features

- Strictly Typed Classes
  - Extends standard ES6+ classes with static typing for properties, methods, constructors, and access modifiers (public, private, protected)
- Type-Only Imports & Modular Code
  - Uses standard ES Modules (import / export), but allows importing types explicitly (which are removed during transpilation, having zero impact on bundle size)

```
import type { User } from './types';
```

- Built-in & Ambient Types
  - Native type definitions for the host environment (e.g., browser/Node.js) - e.g., HTMLElement, HTMLInputElement, HTMLButtonElement, MouseEvent
- Type Assertions (as)
  - Forces a specific type when the compiler lacks context.

```
const input = document.getElementById('name') as HTMLInputElement;
```

TypeScript in This Course

- Adoption in this course
  - Course materials, and baseline starter code primarily use standard JavaScript.
  - Students may adopt TypeScript in their projects as an alternative, requiring further exploration and research
- TypeScript Online Playground
  - Useful environments to experiment with type syntax, test interfaces/generics, and inspect transpiled JavaScript output in real time:

    https://www.typescriptlang.org/play

    https://playcode.io/typescript
- Documentation & Resources
  - Official Handbook: https://www.typescriptlang.org/docs
  - Vue 3 + TypeScript Guide: https://vuejs.org/guide/typescript/overview.html
  - Numerous free tutorials, cheat sheets, and video courses are available across the web (e.g., Total TypeScript, TypeScript Deep Dive, freeCodeCamp).

Bibliography

- Haas Labs Ltd, "RunJS The JavaScript playground for your desktop", https://runjs.app/
- Playcode, "Javascript Playground", https://playcode.io/
- "Can I use...", https://caniuse.com/
- ECMA International, "ECMAScript Language Specification", https://262.ecma-international.org/
- Mozilla Corporation's, "MDN Web Docs", https://developer.mozilla.org/
- Mozilla Corporation's, "JavaScript", https://developer.mozilla.org/docs/Web/JavaScript
- Mozilla Corporation's, "WebAssembly", https://developer.mozilla.org/docs/WebAssembly
- Mozilla Corporation's, "async function", https://developer.mozilla.org/docs/Web/JavaScript/Reference/Statements/async_function
- Mozilla Corporation's, "Document Object Model (DOM)", https://developer.mozilla.org/docs/Web/API/Document_Object_Model
- Google, "Google JavaScript Style Guide", https://google.github.io/styleguide/jsguide.html

Bibliography

- Ilya Kantor, "The JavaScript language", https://javascript.info/js
- Nathan Sebhastian, "Advanced JavaScript Operators – Nullish Coalescing, Optional Chaining, and Destructuring Assignment", https://www.freecodecamp.org/news/javascript-advanced-operators/
- Sarah Okolo, "Mastering JavaScript: optional chaining and nullish coalescing", https://blog.openreplay.com/mastering-javascript-optional-chaining-and-nullish-coalescing/
- Ángel Cereijo, "Diving into Object Cloning: Exploring Alternatives and Limitations", https://amcereijo.medium.com/diving-into-object-cloning-exploring-alternatives-and-limitations-644f0c71315d
- Jecelyn Yeen, "JavaScript Promises for Dummies", https://scotch.io/tutorials/javascript-promises-for-dummies
- Javascript Info, "Async/await", https://javascript.info/async-await

Bibliography

- Diego Salinas Gardón, "The Complete JavaScript Module Bundlers Guide", https://snipcart.com/blog/javascript-module-bundler
- Lin Clark, "ES modules: A cartoon deep-dive", https://hacks.mozilla.org/2018/03/es-modules-a-cartoon-deep-dive/
- VoidZero Inc, "Vite,The Build Tool for the Web", https://vitejs.dev
- Flanagan, D. (2020). JavaScript: The Definitive Guide (7th ed.). O'Reilly.
- Crockford, D. (2008). JavaScript: The Good Parts. O'Reilly.
- Haverbeke, M. (2018). Eloquent JavaScript (3rd ed.). No Starch Press.
- Microsoft, "TypeScript", https://www.typescriptlang.org/
- Microsoft, "TypeScript Documentation", https://www.typescriptlang.org/docs/

The authors acknowledge the use of OpenAI ChatGPT (https://chatgpt.com/) and Google Gemini (https://gemini.google.com) to assist with content development and text editing. The authors retain full responsibility for the contents of this work.